

Oblig 2 3 – Innleggelser

Bilde/skjerm bilde av skjema som er opprett, og hvilke tabeller som ligger i skjemaet

Visning av innhold i de forskjellige tabellene

SCHEMAS

Filter objects

- forelesnign
- oblig**
 - Tables
 - innleggelser
 - leger
 - pasienter
 - provins
 - Views
 - Stored Procedures
 - Functions
- sys
- test1
- world

Table: pasienter

Columns:

- pasient_id** int PK
- fornavn varchar(45)
- etternavn varchar(45)
- kjonn char(1)
- fodselsdag** date PK
- sted varchar(40)
- allergier varchar(50)
- hoyde int
- vekt int
- fk_provins_id** char(2)

	pasient_id	fornavn	etternavn	kjonn	fodselsdag	sted	allergier	hoyde	vekt	fk_provins_id
1	Donald	Waterfield	M	1963-02-12	Barrie	NKA	156	65	ON	
2	Mickey	Baasha	M	1981-05-28	Dundas	Sulfa	185	76	ON	
3	Jiji	Sharma	M	1957-09-05	Hamilton	Penicillin	194	106	ON	
4	Blair	Diaz	M	1967-01-07	Hamilton	NKA	191	104	ON	
5	Charles	Wolfe	M	2017-11-19	Orillia	Penicillin	47	10	ON	
6	Sue	Falcon	F	2017-09-30	Ajax	Penicillin	43	5	ON	
7	Thomas	O'Neill	M	1993-01-31	Burlington	NKA	180	117	ON	
8	Sonny	Beckett	M	1952-12-11	Port Hawkesbury	NKA	174	105	NS	
9	Sister	Spitzer	F	1966-10-15	Toronto	Penicillin	173	95	ON	
10	Cedric	Coltrane	M	1961-11-10	Toronto	NKA	157	61	ON	
11	Hank	Spencer	M	1969-08-10	Peterborough	NKA	158	74	ON	
12	Sara	di Marco	F	1949-04-29	Hamilton	NKA	145	46	ON	
13	Daphne	Seabright	F	1954-11-18	Ancaster	Codeine	146	77	ON	
14	Rick	Bennett	M	1977-01-27	Ancaster	Penicillin	220	95	ON	
15	Amy	Leela	F	1977-06-25	Hamilton	NKA	172	72	ON	
16	Woodv	Bashir	M	1951-11-15	Barrie	Penicillin	153	59	ON	

pasienter 29 x

Table: provins

Columns:

- provins_id** char(2) PK
- provins_navn text

provins_id	provins_navn
AB	Alberta
BC	British Columbia
MB	Manitoba
NB	New Brunswick
NL	Newfoundland and Labrador
NS	Nova Scotia
NT	Northwest Territories
NU	Nunavut
ON	Ontario
PE	Prince Edward Island
QC	Quebec
SK	Saskatchewan
YT	Yukon
NULL	NULL

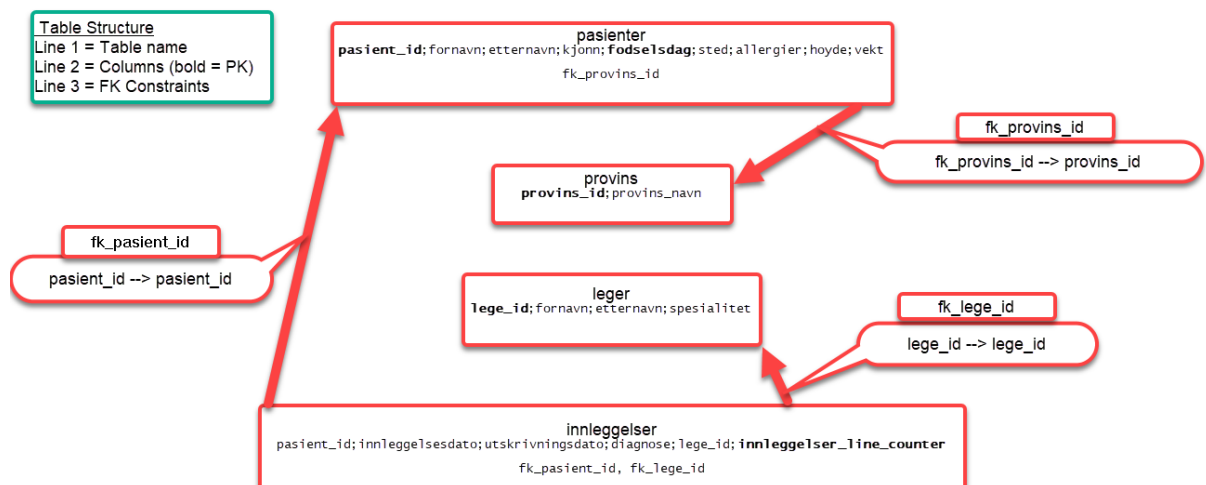
Oblig 2_3 – Innleggelser – Andreas Isaksen

	pasient_id	innleggesdato	utskrivningsdato	diagnose	lege_id	innleggelser_line_counter
	1	2018-11-06	2018-11-08	Ovarian Dermoid-Cyct	21	1
	1	2018-09-20	2018-09-20	Ineffective Breathin Pattern R/T Fluid Accumulatio	24	2
	3	2019-01-24	2019-01-29	Cardiac Arrest	2	3
	3	2018-10-21	2018-10-27	Congestive Heart Failure	8	4
	6	2018-06-13	2018-06-15	Asthma Exacerbation	3	5
	6	2018-11-08	2018-11-09	Uterine Fibroid	22	6
	7	2018-06-24	2018-07-03	Cancer	8	7
	8	2018-09-18	2018-09-21	Amigima	6	8
	9	2019-03-02	2019-03-09	Osteoarthritis	8	9
	9	2018-12-31	2018-12-31	Ruptured Appendicitis	19	10
	10	2018-12-30	2019-01-05	Zenkers Diverticulitis	14	11
	10	2019-02-27	2019-02-27	Lower Quadrant Pain	27	12
	11	2018-12-14	2018-12-16	Prostatectomy	15	13
	12	2019-04-27	2019-05-04	Cerebral Aneurysm Rupture	21	14
	13	2019-04-28	2019-05-01	Renal Failure	1	15
	15	2018-10-01	2018-10-05	Hiatal Hernia	5	16

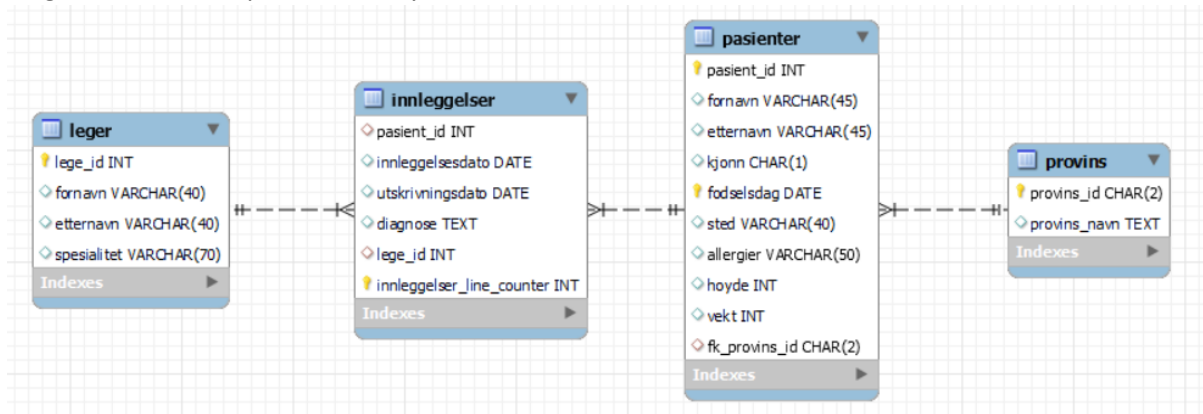
	lege_id	fornavn	etternavn	spesialitet
	1	Claude	Walls	Internist
	2	Joshua	Green	Cardiologist
	3	Miriam	Tregre	General Surgeon
	4	James	Russo	Obstetrician/Gynecologist
	5	Scott	Hill	Gastroenterologist
	6	Tasha	Phillips	Psychiatrist
	7	Hazel	Patterson	Oncologist
	8	Mickey	Duval	Pediatrician
	9	Jon	Nelson	Neurologist
	10	Monica	Singleton	Orthopaedic Surgeon
	11	Douglas	Brooks	Respirologist
	12	Flora	Moore	Cardiovascular Surgeon
	13	Angelica	Noe	Nuclear Medicine
	14	Tyrone	Smart	Gerontologist
	15	Marie	Brinkman	Urologist
	16	Irene	Brooks	Gastroenteroloigist

En skriftlig forklaring av datamodellen, denne kan du opprette ved i Workbench å trykke på "Database" -> "Reverse Engineer". Datmodellen kommer til å inneholde følgende informasjon:

Her startet jeg med å tegne meg en egen datamodell før jeg startet å knytte tabellene mine sammen med FK'er. Dette er den modell skissen jeg tegnet meg:



Med denne løsningen fant jeg at jeg kunne knytte(JOIN) alle tabellene sammen på en måte slik at det vil være mulig å hente ut relevant informasjon på mange måter. Her er skjermbilde av ESS diagrammet som er produsert i MySQL WB:



Hvordan er de forskjellige tabellene koblet sammen?

Først legger jeg ved en illustrasjon av koblingene mine mellom tabellene fra MySQL WB. Videre forklaring ligger under skjermbildet:

```

5 • SELECT
6     CONSTRAINT_NAME,
7     TABLE_NAME,
8     COLUMN_NAME,
9     REFERENCED_TABLE_NAME,
10    REFERENCED_COLUMN_NAME
11 FROM
12     INFORMATION_SCHEMA.KEY_COLUMN_USAGE
13 WHERE
14     CONSTRAINT_SCHEMA = 'oblig'
15     AND CONSTRAINT_NAME LIKE 'fk_%';
16

```

CONSTRAINT_NAME	TABLE_NAME	COLUMN_NAME	REFERENCED_TABLE_NAME	REFERENCED_COLUMN_NAME
fk_innleggelser_leger	innleggelser	lege_id	leger	lege_id
fk_innleggelser_pasienter	innleggelser	pasient_id	pasienter	pasient_id
fk_pasienter_provins	pasienter	fk_provins_id	provins	provins_id

Jeg har da opprettet tre FK'er som knytter alle fire tabeller sammen på hva jeg anså som en logisk måte. Constraint navn er gitt med følgende struktur; nøkkeltipe_fratabell_tiltabell.

- **fk_innleggelser_leger** er en kobling mellom tabellene 'innleggelser' og 'leger'. Her er innleggelser.lege_id kolonnen FK til leger.lege_id kolonnen.
- **fk_innleggelser_pasienter** er en kobling mellom tabellene 'innleggelser' og pasienter. Her er innleggelser.pasient_id FK til pasienter.pasient_id kolonnen.
- **fk_pasienter_provins** er en kobling mellom tabellene 'pasienter' og 'provins'. Her er pasienter.fk_provins_id FK til provins.provins_id. I dette tilfellet er kolonnen pasienter.fk_provins_id opprettet spesifikt som en FK mot provins tabellen.

I hvilken rekkefølge fyller du tabellene med informasjon?

Da innhentet data til de respektive tabellene har kommet fra separate file uten behov for noen direkte sammenknytning før import, har jeg altså ikke sett behov for noen bestemt rekkefølge å importere disse filene på. De har dermed blitt importert basert på hvilke som først har vært tilgjengelige gjennom oppgavenes forløp. I dette tilfellet har rekkefølgen vært følgende:

1. pasienter.csv → pasienter
2. provins.csv → provins
3. innleggelser.csv → innleggelser
4. leger.csv → leger

De eneste direkte endringen jeg har utført i tabellene var å opprette kolonnen `fk_provins_id` i pasienter tabellen, slik at jeg kunne bruke denne som FK mot provins tabellen, og opprettet en 'auto increment' kolonne(`innleggelser_line_counter`) i innleggelser tabellen. Kolonnen 'innleggelser_line_counter' er satt som PK i innleggelser tabellen da denne alltid vil være unik og for at det ikke skal tulle med FK'er som er satt opp mellom tabeller ved at jeg har en sammensatt PK i tabellen.

En skriftlig forklaring på hva hver spørring skal hente av informasjon og en skjermdump av resultatet av hver spørring.

1. Vis antall innleggelser

```
SELECT COUNT(innleggelser_line_counter) AS antall_innleggelser
FROM innleggelser;
```

	antall_innleggelser
▶	567

2. Vis alle innleggelser hvor en pasient ble innlagt og utskrevet samme dag

```
SELECT concat(pasienter.fornavn, ' ', pasienter.etternavn) AS pasient,
       concat(innleggelser.innleggelsesdato, ' - ',
innleggelser.utskrivningsdato) AS 'INN/UT'
FROM innleggelser
JOIN pasienter ON innleggelser.pasient_id = pasienter.pasient_id
WHERE innleggelsesdato = utskrivningsdato
ORDER BY innleggelsesdato;
```

	pasient	INN/UT
▶	Thursday Ernst	2018-06-07 - 2018-06-07
	Michael Jeter	2018-06-14 - 2018-06-14
	Natsumi Weasley	2018-06-17 - 2018-06-17
	Haiku Shaft	2018-07-03 - 2018-07-03
	Vicki Cerva	2018-07-14 - 2018-07-14
	Billy Jones	2018-07-17 - 2018-07-17
	Naoto Stewart	2018-08-08 - 2018-08-08
	Gil Sigel	2018-08-13 - 2018-08-13
	Joyce Bosch	2018-08-25 - 2018-08-25
	Zoe Miller	2018-08-30 - 2018-08-30

3. Vis pasient_id og antallet innleggelser til pasient 179

```
SELECT pasienter.pasient_id,  
       COUNT(innleggelser.pasient_id) AS antall_innleggelser  
FROM pasienter  
JOIN innleggelser ON pasienter.pasient_id = innleggelser.pasient_id  
WHERE innleggelser.pasient_id = 179;
```

	pasient_id	antall_innleggelser
▶	179	2